AI For Math Seminar, ZJU

Lec 06: Formalize and Verify Algorithm and Theorem

Mulikas

Outline

Outline

Introduction

- “Everyone could claim he/she has a correct proof of the Riemann Hypothesis, but the first step is to finish writing the paper.”

Introduction

- "Everyone could claim he/she has a correct proof of the Riemann Hypothesis, but the first step is to finish writing the paper."
- "Talk is cheap, show me the code." (Linus Torvalds)

Introduction

- "Everyone could claim he/she has a correct proof of the Riemann Hypothesis, but the first step is to finish writing the paper."
- "Talk is cheap, show me the code." (Linus Torvalds)
- Similarly, in software engineering, we can write an infinite number of functions that receive the same type of input and produce some output. How can we rigorously know which one is correct?

Introduction

- "Everyone could claim he/she has a correct proof of the Riemann Hypothesis, but the first step is to finish writing the paper."
- "Talk is cheap, show me the code." (Linus Torvalds)
- Similarly, in software engineering, we can write an infinite number of functions that receive the same type of input and produce some output. How can we rigorously know which one is correct?
- In this section, we first recall foundational knowledge in Computer Science, then we translate these concepts into Lean 4, learning the practical skills to verify real projects.

Correctness and Termination of an Algorithm 1

The fundamental question: What does it mean for an algorithm to be "correct"?

- Under the Curry-Howard Isomorphism, we already know that an algorithm corresponds to the construction of a mathematical theorem. So the question becomes: what is the correctness of a theorem?

Correctness and Termination of an Algorithm 1

The fundamental question: What does it mean for an algorithm to be "correct"?

- Under the Curry-Howard Isomorphism, we already know that an algorithm corresponds to the construction of a mathematical theorem. So the question becomes: what is the correctness of a theorem?
- In the List Sorting problem, we want a procedure to receive an arbitrary-length finite list and return a sorted list. Here, "correctness" is formally defined as maintaining the permutation of elements while satisfying a mathematical ordering property.

Correctness and Termination of an Algorithm 1

The fundamental question: What does it mean for an algorithm to be "correct"?

- Under the Curry-Howard Isomorphism, we already know that an algorithm corresponds to the construction of a mathematical theorem. So the question becomes: what is the correctness of a theorem?
- In the List Sorting problem, we want a procedure to receive an arbitrary-length finite list and return a sorted list. Here, "correctness" is formally defined as maintaining the permutation of elements while satisfying a mathematical ordering property.
- Therefore, we need to define correctness manually. In Lean, we encode the human intent of "correctness" as a mathematically rigorous **Proposition**.

Correctness and Termination of an Algorithm 2

Let's review the classical CS definitions and prepare to map them to Lean.

Definition (Specification and Total Correctness)

Assume the task: Given a list of numbers, compute their sum.

Correctness and Termination of an Algorithm 2

Let's review the classical CS definitions and prepare to map them to Lean.

Definition (Specification and Total Correctness)

Assume the task: Given a list of numbers, compute their sum.

- **Specification**: The **Input** is a list of natural numbers, and the **Output** is a single natural number. The **Final Condition** (Post-condition) is that the output strictly equals the mathematical sum of the elements.

Correctness and Termination of an Algorithm 2

Let's review the classical CS definitions and prepare to map them to Lean.

Definition (Specification and Total Correctness)

Assume the task: Given a list of numbers, compute their sum.

- **Specification**: The **Input** is a list of natural numbers, and the **Output** is a single natural number. The **Final Condition** (Post-condition) is that the output strictly equals the mathematical sum of the elements.
- **Total Correctness**: For any given specification, we say an algorithm is **totally correct** if and only if:

Correctness and Termination of an Algorithm 2

Let's review the classical CS definitions and prepare to map them to Lean.

Definition (Specification and Total Correctness)

Assume the task: Given a list of numbers, compute their sum.

- **Specification**: The **Input** is a list of natural numbers, and the **Output** is a single natural number. The **Final Condition** (Post-condition) is that the output strictly equals the mathematical sum of the elements.
- **Total Correctness**: For any given specification, we say an algorithm is **totally correct** if and only if:
  1. It always terminates (Halting).

Correctness and Termination of an Algorithm 2

Let's review the classical CS definitions and prepare to map them to Lean.

Definition (Specification and Total Correctness)

Assume the task: Given a list of numbers, compute their sum.

- **Specification**: The **Input** is a list of natural numbers, and the **Output** is a single natural number. The **Final Condition** (Post-condition) is that the output strictly equals the mathematical sum of the elements.
- **Total Correctness**: For any given specification, we say an algorithm is **totally correct** if and only if:
  1. It always terminates (Halting).
  2. It returns the correct result according to the specification.

Correctness and Termination of an Algorithm 2

Let's review the classical CS definitions and prepare to map them to Lean.

Definition (Specification and Total Correctness)

Assume the task: Given a list of numbers, compute their sum.

- **Specification**: The **Input** is a list of natural numbers, and the **Output** is a single natural number. The **Final Condition** (Post-condition) is that the output strictly equals the mathematical sum of the elements.
- **Total Correctness**: For any given specification, we say an algorithm is **totally correct** if and only if:
  1. It always terminates (Halting).
  2. It returns the correct result according to the specification.
- As a result, to formally verify an algorithm, our proof burden is twofold: we must prove **Termination** and **Semantic Correctness**.

Exercise in Lean: Formulating the Problem

In Lean, every function is inherently required to be totally correct. You simply cannot compile a function that might infinitely loop without an escape hatch. Let's tackle Correctness first.

- **Translating the Specification**: Because Lean allows us to use Propositional Logic as a type (Dependent Type Theory), we declare "correctness" through a Theorem signature. What we must check is whether our Theorem actually captures the meaning we want.

Exercise in Lean: Formulating the Problem

In Lean, every function is inherently required to be totally correct. You simply cannot compile a function that might infinitely loop without an escape hatch. Let's tackle Correctness first.

- **Translating the Specification**: Because Lean allows us to use Propositional Logic as a type (Dependent Type Theory), we declare "correctness" through a Theorem signature. What we must check is whether our Theorem actually captures the meaning we want.
- **Exercise**: Write a recursive function to sum up an arbitrary `List Nat`. (We ignore hardware overflow limits here; Lean's `Nat` is arbitrarily large).

Exercise in Lean: Formulating the Problem

In Lean, every function is inherently required to be totally correct. You simply cannot compile a function that might infinitely loop without an escape hatch. Let's tackle Correctness first.

- **Translating the Specification**: Because Lean allows us to use Propositional Logic as a type (Dependent Type Theory), we declare "correctness" through a Theorem signature. What we must check is whether our Theorem actually captures the meaning we want.
- **Exercise**: Write a recursive function to sum up an arbitrary `List Nat`. (We ignore hardware overflow limits here; Lean's `Nat` is arbitrarily large).
- Once written, how do we mathematically prove it behaves like a sum?

Exercise in Lean: Writing the Code

Exercise in Lean: Writing the Code

First, we define the computational procedure. We use structural recursion over the list.

```
def listSum (xs : List Nat) : Nat :=
  match xs with
  | [] => 0
  | y :: ys => y + listSum ys
```

Exercise in Lean: Writing the Code

First, we define the computational procedure. We use structural recursion over the list.

```
def listSum (xs : List Nat) : Nat :=
  match xs with
  | [] => 0
  | y :: ys => y + listSum ys
```

Next, we must invent a **Specification** (Theorem). A good way to prove a sum is correct is to prove it distributes over list concatenation (++):

```
theorem listSum_append (xs ys : List Nat) :
  listSum (xs ++ ys) = listSum xs + listSum ys := by
  -- Proof goes here
```

Solution Strategy: Induction

Our Task: Prove the correctness of the hypothesis given our defined type.

- Since computational data structures (like Lists and Nats) are constructed from finite, discrete rules, our primary weapon is **Structural Induction**.

Solution Strategy: Induction

Our Task: Prove the correctness of the hypothesis given our defined type.

- Since computational data structures (like Lists and Nats) are constructed from finite, discrete rules, our primary weapon is **Structural Induction**.
- For this specific problem, we can just use the default constructors of `List`: the empty list (`nil`) and the appended element (`cons`).

Solution Strategy: Induction

Our Task: Prove the correctness of the hypothesis given our defined type.

- Since computational data structures (like Lists and Nats) are constructed from finite, discrete rules, our primary weapon is **Structural Induction**.
- For this specific problem, we can just use the default constructors of `List`: the empty list (`nil`) and the appended element (`cons`).
- In more complex scenarios, our strategy is to use the `cases` or `match` tactics to logically divide the type into exhaustive branches, and finish the proof for each branch.

Solution Strategy: Induction

Our Task: Prove the correctness of the hypothesis given our defined type.

- Since computational data structures (like Lists and Nats) are constructed from finite, discrete rules, our primary weapon is **Structural Induction**.
- For this specific problem, we can just use the default constructors of `List`: the empty list (`nil`) and the appended element (`cons`).
- In more complex scenarios, our strategy is to use the `cases` or `match` tactics to logically divide the type into exhaustive branches, and finish the proof for each branch.
- For example, using a theorem like "Every natural number is either even or odd" to introduce new hypotheses, allowing us to traverse deeper into number theory proofs.

Solution Execution: The Proof

Solution Execution: The Proof

We apply structural induction on the first list, `xs`.

```
theorem listSum_append (xs ys : List Nat) :
    listSum (xs ++ ys) = listSum xs + listSum ys := by
  induction xs with
  | nil =>
    -- Base case: listSum ([] ++ ys) = listSum [] + listSum ys
    -- This is trivially true by definition of listSum.
    rfl
  | cons head tail ih =>
    -- Inductive step: we assume the theorem holds for `tail` (ih).
    calc
      listSum ((head :: tail) ++ ys)
        = head + listSum (tail ++ ys)        := by rfl
      _ = head + (listSum tail + listSum ys) := by rw [ih]
      _ = (head + listSum tail) + listSum ys := by omega
      _ = listSum (head :: tail) + listSum ys := by rfl
```

Correctness and Termination of an Algorithm 3

- We have successfully proven the semantic **Correctness** of our algorithm using induction.

Correctness and Termination of an Algorithm 3

- We have successfully proven the semantic **Correctness** of our algorithm using induction.
- However, recall our definition of Total Correctness: the algorithm must also **Terminate**.

Correctness and Termination of an Algorithm 3

- We have successfully proven the semantic **Correctness** of our algorithm using induction.
- However, recall our definition of Total Correctness: the algorithm must also **Terminate**.
- Why didn't Lean ask us for a termination proof for `listSum`? Because Lean automatically detected that `tail` is strictly smaller than `head :: tail` (Structural Recursion).

Correctness and Termination of an Algorithm 3

- We have successfully proven the semantic **Correctness** of our algorithm using induction.
- However, recall our definition of Total Correctness: the algorithm must also **Terminate**.
- Why didn't Lean ask us for a termination proof for `listSum`? Because Lean automatically detected that `tail` is strictly smaller than `head :: tail` (Structural Recursion).
- The final goal of formal verification is to handle cases where algorithms do *not* trivially shrink.

Correctness and Termination of an Algorithm 3

- We have successfully proven the semantic **Correctness** of our algorithm using induction.
- However, recall our definition of Total Correctness: the algorithm must also **Terminate**.
- Why didn't Lean ask us for a termination proof for `listSum`? Because Lean automatically detected that `tail` is strictly smaller than `head :: tail` (Structural Recursion).
- The final goal of formal verification is to handle cases where algorithms do *not* trivially shrink.
- In Lean, proving termination is equivalent to writing a well-defined function that passes the kernel's strictly descending metrics without a compiler error. This brings us to Section 2.

Outline

The Philosophy of Totality

- **The Mainstream View:** In languages like C, Java, or Python, functions are *partial*. They might return a value, they might throw an exception, or they might loop forever.

The Philosophy of Totality

- **The Mainstream View:** In languages like C, Java, or Python, functions are *partial*. They might return a value, they might throw an exception, or they might loop forever.
- The Halting Problem states that no compiler can statically guarantee termination for all programs in a Turing-complete language.

The Philosophy of Totality

- **The Mainstream View:** In languages like C, Java, or Python, functions are *partial*. They might return a value, they might throw an exception, or they might loop forever.
- The Halting Problem states that no compiler can statically guarantee termination for all programs in a Turing-complete language.
- **The Lean 4 View:** Lean strictly requires *Totality*. Every defined function **must** terminate in a finite number of steps for all possible inputs.

The Philosophy of Totality

- **The Mainstream View:** In languages like C, Java, or Python, functions are *partial*. They might return a value, they might throw an exception, or they might loop forever.
- The Halting Problem states that no compiler can statically guarantee termination for all programs in a Turing-complete language.
- **The Lean 4 View:** Lean strictly requires *Totality*. Every defined function **must** terminate in a finite number of steps for all possible inputs.
- **Why?** Because of the Curry-Howard Isomorphism.

Why Infinite Loops Destroy Mathematics

Why Infinite Loops Destroy Mathematics

If Lean allowed non-terminating functions, the logical foundation of the entire system would instantly collapse.

Why Infinite Loops Destroy Mathematics

If Lean allowed non-terminating functions, the logical foundation of the entire system would instantly collapse.

Imagine Lean permitted this infinite loop:

```
-- Hypothetical code (Lean will reject this!)
def paradox : False := paradox
```

Why Infinite Loops Destroy Mathematics

If Lean allowed non-terminating functions, the logical foundation of the entire system would instantly collapse.

Imagine Lean permitted this infinite loop:

```
-- Hypothetical code (Lean will reject this!)
def paradox : False := paradox
```

- By Curry-Howard, providing a term of type `False` means you have proven a contradiction.

Why Infinite Loops Destroy Mathematics

If Lean allowed non-terminating functions, the logical foundation of the entire system would instantly collapse.

Imagine Lean permitted this infinite loop:

```
-- Hypothetical code (Lean will reject this!)
def paradox : False := paradox
```

- By Curry-Howard, providing a term of type `False` means you have proven a contradiction.
- If you can prove `False`, you can use the Principle of Explosion (*ex falso quodlibet*) to prove $1 = 0$, $P \wedge \sim P$, or any other absurdity.

Why Infinite Loops Destroy Mathematics

If Lean allowed non-terminating functions, the logical foundation of the entire system would instantly collapse.

Imagine Lean permitted this infinite loop:

```
-- Hypothetical code (Lean will reject this!)
def paradox : False := paradox
```

- By Curry-Howard, providing a term of type `False` means you have proven a contradiction.
- If you can prove `False`, you can use the Principle of Explosion (*ex falso quodlibet*) to prove $1 = 0$, $P \wedge \sim P$, or any other absurdity.
- **Conclusion:** Proving termination is not just a performance optimization; it is a fundamental requirement for mathematical soundness.

The Happy Path: Structural Recursion

The Happy Path: Structural Recursion

For most data structures, Lean proves termination automatically by observing **Structural Decreasing**.

```
def list_len {A : Type} (xs : List A) : Nat :=
  match xs with
  | [] => 0
  | _ :: tail => 1 + list_len tail
```

The Happy Path: Structural Recursion

For most data structures, Lean proves termination automatically by observing **Structural Decreasing**.

```
def list_len {A : Type} (xs : List A) : Nat :=
  match xs with
  | [] => 0
  | _ :: tail => 1 + list_len tail
```

- `tail` is a structurally strictly smaller sub-component of `_ :: tail`.

The Happy Path: Structural Recursion

For most data structures, Lean proves termination automatically by observing **Structural Decreasing**.

```
def list_len {A : Type} (xs : List A) : Nat :=
  match xs with
  | [] => 0
  | _ :: tail => 1 + list_len tail
```

- `tail` is a structurally strictly smaller sub-component of `_ :: tail`.
- Inductive types in Lean are strictly finite trees. You cannot descend into a finite tree infinitely.

The Happy Path: Structural Recursion

For most data structures, Lean proves termination automatically by observing **Structural Decreasing**.

```
def list_len {A : Type} (xs : List A) : Nat :=
  match xs with
  | [] => 0
  | _ :: tail => 1 + list_len tail
```

- `tail` is a structurally strictly smaller sub-component of `_ :: tail`.
- Inductive types in Lean are strictly finite trees. You cannot descend into a finite tree infinitely.
- Lean's internal heuristics accept this without any manual proof.

When Structural Recursion Fails

When Structural Recursion Fails

What happens when an algorithm shrinks its input *mathematically*, rather than *structurally*?

```
def log2 (n : Nat) : Nat :=
  if n < 2 then
    0
  else
    1 + log2 (n / 2)
```

When Structural Recursion Fails

What happens when an algorithm shrinks its input *mathematically*, rather than *structurally*?

```
def log2 (n : Nat) : Nat :=
  if n < 2 then
    0
  else
    1 + log2 (n / 2)
```

Compiler Error:

```
fail to show termination for log2
unproved application: log2 (n / 2)
```

When Structural Recursion Fails

What happens when an algorithm shrinks its input *mathematically*, rather than *structurally*?

```
def log2 (n : Nat) : Nat :=
  if n < 2 then
    0
  else
    1 + log2 (n / 2)
```

Compiler Error:

```
fail to show termination for log2
unproved application: log2 (n / 2)
```

Lean sees `n / 2`. To the compiler, this is an arbitrary function call. It does not natively know that division by 2 produces a smaller integer.

The Solution: Well-Founded Relations

- To prove termination for non-structural algorithms, we use **Well-Founded Recursion**.

The Solution: Well-Founded Relations

- To prove termination for non-structural algorithms, we use **Well-Founded Recursion**.
- **Mathematical Definition:** A relation $\prec$ on a set S is well-founded if every non-empty subset of S has a minimal element with respect to $\prec$.

The Solution: Well-Founded Relations

- To prove termination for non-structural algorithms, we use **Well-Founded Recursion**.
- **Mathematical Definition:** A relation $\prec$ on a set S is well-founded if every non-empty subset of S has a minimal element with respect to $\prec$.
- **Equivalently:** There are no countably infinite descending chains $x_0 \succ x_1 \succ x_2 \succ \ldots$

The Solution: Well-Founded Relations

- To prove termination for non-structural algorithms, we use **Well-Founded Recursion**.
- **Mathematical Definition:** A relation $\prec$ on a set S is well-founded if every non-empty subset of S has a minimal element with respect to $\prec$.
- **Equivalently:** There are no countably infinite descending chains $x_0 \succ x_1 \succ x_2 \succ \ldots$
- **The Gold Standard:** The strictly-less-than relation $(<)$ on Natural Numbers (`Nat`). Because you cannot go below 0, any sequence of strictly decreasing natural numbers must eventually stop.

Defining the Measure: `termination_by`

Defining the Measure: `termination_by`

We guide Lean by defining a **Measure** — a mapping from the function's arguments to a `Nat` that strictly decreases.

```
def log2 (n : Nat) : Nat :=
  if h : n < 2 then
    0
  else
    1 + log2 (n / 2)
termination_by n
```

Defining the Measure: `termination_by`

We guide Lean by defining a **Measure** — a mapping from the function's arguments to a `Nat` that strictly decreases.

```
def log2 (n : Nat) : Nat :=
  if h : n < 2 then
    0
  else
    1 + log2 (n / 2)
termination_by n
```

- `termination_by n` tells Lean: "Check if the natural number `n` strictly decreases on every recursive call."

Defining the Measure: `termination_by`

We guide Lean by defining a **Measure** — a mapping from the function's arguments to a `Nat` that strictly decreases.

```
def log2 (n : Nat) : Nat :=
  if h : n < 2 then
    0
  else
    1 + log2 (n / 2)
termination_by n
```

- `termination_by n` tells Lean: "Check if the natural number `n` strictly decreases on every recursive call."
- Notice the `h : n < 2`. We bind the condition to a hypothesis!

Defining the Measure: `termination_by`

We guide Lean by defining a **Measure** — a mapping from the function's arguments to a `Nat` that strictly decreases.

```
def log2 (n : Nat) : Nat :=
  if h : n < 2 then
    0
  else
    1 + log2 (n / 2)
termination_by n
```

- `termination_by n` tells Lean: "Check if the natural number `n` strictly decreases on every recursive call."
- Notice the `h : n < 2`. We bind the condition to a hypothesis!
- In the `else` branch, Lean's context now contains `Not (n < 2)`, meaning $n \geq 2$. Lean's automated arithmetic can now prove $(n/2) < n$.

When Automation Fails

When Automation Fails

Sometimes, the mathematical descent is too complex for Lean's automatic solvers. Consider division by repeated subtraction.

```
def slow_div (n k : Nat) : Nat :=
  if h0 : k = 0 then 0
  else if h1 : n < k then 0
  else 1 + slow_div (n - k) k
termination_by n
```

When Automation Fails

Sometimes, the mathematical descent is too complex for Lean's automatic solvers. Consider division by repeated subtraction.

```
def slow_div (n k : Nat) : Nat :=
  if h0 : k = 0 then 0
  else if h1 : n < k then 0
  else 1 + slow_div (n - k) k
termination_by n
```

This fails to compile! Lean generates a **Termination Goal**:

```
|- n - k < n
```

When Automation Fails

Sometimes, the mathematical descent is too complex for Lean's automatic solvers. Consider division by repeated subtraction.

```
def slow_div (n k : Nat) : Nat :=
  if h0 : k = 0 then 0
  else if h1 : n < k then 0
  else 1 + slow_div (n - k) k
termination_by n
```

This fails to compile! Lean generates a **Termination Goal**:
`|- n - k < n`

Lean cannot prove this blindly. If $k = 0$, then $n - 0$ is not less than n. We must intervene manually.

Manual Overrides: `decreasing_by`

Manual Overrides: `decreasing_by`

We can append a `decreasing_by` tactic block. This opens a standard proof environment specifically for the termination goal.

```
def slow_div (n k : Nat) : Nat :=
  if h0 : k = 0 then 0
  else if h1 : n < k then 0
  else 1 + slow_div (n - k) k
termination_by n
decreasing_by
  -- Context contains:
  -- h0 : Not (k = 0),  so k >= 1
  -- h1 : Not (n < k),  so n >= k
  -- Goal: n - k < n
  omega
```

Manual Overrides: `decreasing_by`

We can append a `decreasing_by` tactic block. This opens a standard proof environment specifically for the termination goal.

```
def slow_div (n k : Nat) : Nat :=
  if h0 : k = 0 then 0
  else if h1 : n < k then 0
  else 1 + slow_div (n - k) k
termination_by n
decreasing_by
  -- Context contains:
  -- h0 : Not (k = 0),  so k >= 1
  -- h1 : Not (n < k),  so n >= k
  -- Goal: n - k < n
  omega
```

The Power of Context: Because we saved the branch conditions as `h0` and `h1`, `omega` sees that $k \geq 1$ and effortlessly proves $n - k < n$.

Live Lean: `D6_Termination.lean`

Advanced Measures: Beyond Arguments

Advanced Measures: Beyond Arguments

The measure doesn't have to be a direct parameter. It can be a computed expression representing the "remaining work".

```
-- Traversing an array from index `i` to the end
def process {A : Type} (arr : Array A) (i : Nat) : Nat :=
  if h : i < arr.size then
    1 + process arr (i + 1)
  else
    0
-- The parameter `i` is INCREASING, so it can't be the measure!
termination_by arr.size - i
decreasing_by
  -- Goal: arr.size - (i + 1) < arr.size - i
  omega
```

Advanced Measures: Beyond Arguments

The measure doesn't have to be a direct parameter. It can be a computed expression representing the "remaining work".

```
-- Traversing an array from index `i` to the end
def process {A : Type} (arr : Array A) (i : Nat) : Nat :=
  if h : i < arr.size then
    1 + process arr (i + 1)
  else
    0
-- The parameter `i` is INCREASING, so it can't be the measure!
termination_by arr.size - i
decreasing_by
  -- Goal: arr.size - (i + 1) < arr.size - i
  omega
```

Insight: We map the state to a strictly decreasing distance: `arr.size - i`.

Multi-Variable Recursion

Multi-Variable Recursion

What happens when algorithms decrease alternately across multiple parameters? The classic example is the **Ackermann function**.

```
def ack (m n : Nat) : Nat :=
  match m, n with
  | 0, n => n + 1
  | m' + 1, 0 => ack m' 1
  | m' + 1, n' + 1 => ack m' (ack (m' + 1) n')
```

Multi-Variable Recursion

What happens when algorithms decrease alternately across multiple parameters? The classic example is the **Ackermann function**.

```
def ack (m n : Nat) : Nat :=
  match m, n with
  | 0, n => n + 1
  | m' + 1, 0 => ack m' 1
  | m' + 1, n' + 1 => ack m' (ack (m' + 1) n')
```

Notice the chaotic parameter flow:

- Sometimes m goes down, and n resets.

Multi-Variable Recursion

What happens when algorithms decrease alternately across multiple parameters? The classic example is the **Ackermann function**.

```
def ack (m n : Nat) : Nat :=
  match m, n with
  | 0, n => n + 1
  | m' + 1, 0 => ack m' 1
  | m' + 1, n' + 1 => ack m' (ack (m' + 1) n')
```

Notice the chaotic parameter flow:

- Sometimes m goes down, and n resets.
- Sometimes m stays the same, and n goes down.

Multi-Variable Recursion

What happens when algorithms decrease alternately across multiple parameters? The classic example is the **Ackermann function**.

```
def ack (m n : Nat) : Nat :=
  match m, n with
  | 0, n => n + 1
  | m' + 1, 0 => ack m' 1
  | m' + 1, n' + 1 => ack m' (ack (m' + 1) n')
```

Notice the chaotic parameter flow:

- Sometimes m goes down, and n resets.
- Sometimes m stays the same, and n goes down.
- Sometimes n turns into a massive recursive call!

Lexicographic Ordering (Dictionary Order)

- To prove `ack` terminates, Lean uses a **Lexicographic Order** across a tuple of variables.

Lexicographic Ordering (Dictionary Order)

- To prove `ack` terminates, Lean uses a **Lexicographic Order** across a tuple of variables.
- Think of alphabetical sorting: "apple" comes before "apply". You check the first letter. If it ties, you check the second.

Lexicographic Ordering (Dictionary Order)

- To prove `ack` terminates, Lean uses a **Lexicographic Order** across a tuple of variables.
- Think of alphabetical sorting: "apple" comes before "apply". You check the first letter. If it ties, you check the second.
- **Math Definition for pairs** (m, n)**:**
  $(m_2, n_2) \prec (m_1, n_1)$ if and only if:

  $m_2 < m_1$ **OR** $(m_2 = m_1$ **AND** $n_2 < n_1)$

Lexicographic Ordering (Dictionary Order)

- To prove `ack` terminates, Lean uses a **Lexicographic Order** across a tuple of variables.
- Think of alphabetical sorting: "apple" comes before "apply". You check the first letter. If it ties, you check the second.
- **Math Definition for pairs** (m, n)**:**
  $(m_2, n_2) \prec (m_1, n_1)$ if and only if:

  $m_2 < m_1$ **OR** $(m_2 = m_1$ **AND** $n_2 < n_1)$
- As long as the tuple strictly decreases under this definition, infinite loops are mathematically impossible.

Syntax for Lexicographic Measures

Syntax for Lexicographic Measures

To trigger Lexicographic ordering, we pass multiple identifiers to `termination_by`.

```
def ack (m n : Nat) : Nat :=
  match m, n with
  | 0, n => n + 1
  | m' + 1, 0 => ack m' 1
  | m' + 1, n' + 1 => ack m' (ack (m' + 1) n')
termination_by m n
```

Deep Dive: Name Binding vs. Positional Priority

Deep Dive: Name Binding vs. Positional Priority

How does Lean parse `termination_by m n`?

- **1. Binding by Name:** Lean maps the identifiers `m` and `n` to the function parameters. (If you wrote `termination_by x y`, it would fail because those names don't exist).

Deep Dive: Name Binding vs. Positional Priority

How does Lean parse `termination_by m n`?

- **1. Binding by Name:** Lean maps the identifiers `m` and `n` to the function parameters. (If you wrote `termination_by x y`, it would fail because those names don't exist).
- **2. Priority by Position (Crucial):** The left-to-right order in the clause determines mathematical priority.

Deep Dive: Name Binding vs. Positional Priority

How does Lean parse `termination_by m n`?

- **1. Binding by Name:** Lean maps the identifiers `m` and `n` to the function parameters. (If you wrote `termination_by x y`, it would fail because those names don't exist).
- **2. Priority by Position (Crucial):** The left-to-right order in the clause determines mathematical priority.
- Because `m` is written first, Lean will check if $m_{new} < m_{old}$.

Deep Dive: Name Binding vs. Positional Priority

How does Lean parse `termination_by m n`?

- **1. Binding by Name:** Lean maps the identifiers `m` and `n` to the function parameters. (If you wrote `termination_by x y`, it would fail because those names don't exist).
- **2. Priority by Position (Crucial):** The left-to-right order in the clause determines mathematical priority.
- Because `m` is written first, Lean will check if $m_{new} < m_{old}$.
- **If m strictly decreased, Lean IMMEDIATELY declares termination success** and totally ignores what happened to n.

Deep Dive: Name Binding vs. Positional Priority

How does Lean parse `termination_by m n`?

- **1. Binding by Name:** Lean maps the identifiers `m` and `n` to the function parameters. (If you wrote `termination_by x y`, it would fail because those names don't exist).
- **2. Priority by Position (Crucial):** The left-to-right order in the clause determines mathematical priority.
- Because `m` is written first, Lean will check if $m_{new} < m_{old}$.
- **If m strictly decreased, Lean IMMEDIATELY declares termination success** and totally ignores what happened to n.
- Only if m is exactly equal, will Lean look at n.

Tracing the Ackermann Termination (1)

Let's trace the hardest branch: `| m' + 1, n' + 1 => ack m' (ack (m' + 1) n')`

Tracing the Ackermann Termination (1)

Let's trace the hardest branch: `| m' + 1, n' + 1 => ack m' (ack (m' + 1) n')`

The Inner Call: `ack (m' + 1) n'`

Tracing the Ackermann Termination (1)

Let's trace the hardest branch: `| m' + 1, n' + 1 => ack m' (ack (m' + 1) n')`

The Inner Call: `ack (m' + 1) n'`

- Lean checks parameter 1 (`m`): Old was $m' + 1$, New is $m' + 1$. **Tie.**

Tracing the Ackermann Termination (1)

Let's trace the hardest branch: `| m' + 1, n' + 1 => ack m' (ack (m' + 1) n')`

The Inner Call: `ack (m' + 1) n'`

- Lean checks parameter 1 (`m`): Old was $m' + 1$, New is $m' + 1$. **Tie.**
- Because of the tie, Lean checks parameter 2 (`n`): Old was $n' + 1$, New is n'.

Tracing the Ackermann Termination (1)

Let's trace the hardest branch: `| m' + 1, n' + 1 => ack m' (ack (m' + 1) n')`

The Inner Call: `ack (m' + 1) n'`

- Lean checks parameter 1 (`m`): Old was $m' + 1$, New is $m' + 1$. **Tie.**
- Because of the tie, Lean checks parameter 2 (`n`): Old was $n' + 1$, New is n'.
- Since $n' < n' + 1$, the inner call terminates.

Tracing the Ackermann Termination (2)

The Outer Call: `ack m' (...)`

Tracing the Ackermann Termination (2)

The Outer Call: `ack m' (...)`

- Lean checks parameter 1 (`m`): Old was $m' + 1$, New is m'.

Tracing the Ackermann Termination (2)

The Outer Call: `ack m' (...)`

- Lean checks parameter 1 (`m`): Old was $m' + 1$, New is m'.
- Since $m' < m' + 1$, it strictly decreased.

Tracing the Ackermann Termination (2)

The Outer Call: `ack m' (...)`

- Lean checks parameter 1 (`m`): Old was $m' + 1$, New is m'.
- Since $m' < m' + 1$, it strictly decreased.
- **Lean stops checking.**

Tracing the Ackermann Termination (2)

The Outer Call: `ack m' (...)`

- Lean checks parameter 1 (`m`): Old was $m' + 1$, New is m'.
- Since $m' < m' + 1$, it strictly decreased.
- **Lean stops checking.**
- It completely ignores parameter 2. This is brilliant because parameter 2 is `ack (m' + 1) n'`, which is a massive number that Lean cannot statically evaluate!

What if we reversed the measure?

What if we reversed the measure?

What if a student writes `termination_by n m`?

```
def ack_broken (m n : Nat) : Nat :=
  match m, n with
  | 0, n => n + 1
  | m' + 1, 0 => ack_broken m' 1
  | m' + 1, n' + 1 => ack_broken m' (ack_broken (m' + 1) n')
termination_by n m  -- PRIORITY REVERSED!
```

What if we reversed the measure?

What if a student writes `termination_by n m`?

```
def ack_broken (m n : Nat) : Nat :=
  match m, n with
  | 0, n => n + 1
  | m' + 1, 0 => ack_broken m' 1
  | m' + 1, n' + 1 => ack_broken m' (ack_broken (m' + 1) n')
termination_by n m  -- PRIORITY REVERSED!
```

This fails completely.

- For the outer call, Lean checks n first.

What if we reversed the measure?

What if a student writes `termination_by n m`?

```
def ack_broken (m n : Nat) : Nat :=
  match m, n with
  | 0, n => n + 1
  | m' + 1, 0 => ack_broken m' 1
  | m' + 1, n' + 1 => ack_broken m' (ack_broken (m' + 1) n')
termination_by n m  -- PRIORITY REVERSED!
```

This fails completely.

- For the outer call, Lean checks n first.
- Old n was $n' + 1$. New n is `ack_broken (m' + 1) n'`.

What if we reversed the measure?

What if a student writes `termination_by n m`?

```
def ack_broken (m n : Nat) : Nat :=
  match m, n with
  | 0, n => n + 1
  | m' + 1, 0 => ack_broken m' 1
  | m' + 1, n' + 1 => ack_broken m' (ack_broken (m' + 1) n')
termination_by n m  -- PRIORITY REVERSED!
```

This fails completely.

- For the outer call, Lean checks n first.
- Old n was $n' + 1$. New n is `ack_broken (m' + 1) n'`.
- Lean cannot prove that the output of Ackermann is smaller than $n' + 1$. The termination proof crashes.

Mutual Recursion (A Brief Mention)

- What if Function A calls Function B, and Function B calls Function A?

Mutual Recursion (A Brief Mention)

- What if Function A calls Function B, and Function B calls Function A?
- Lean supports **Mutual Recursion** using the `mutual` keyword.

Mutual Recursion (A Brief Mention)

- What if Function A calls Function B, and Function B calls Function A?
- Lean supports **Mutual Recursion** using the `mutual` keyword.
- The termination proof requires finding a unified measure that decreases across *both* functions.

Mutual Recursion (A Brief Mention)

- What if Function A calls Function B, and Function B calls Function A?
- Lean supports **Mutual Recursion** using the `mutual` keyword.
- The termination proof requires finding a unified measure that decreases across *both* functions.
- *(For the sake of time in this seminar, we omit the deep dive into mutual termination, but it relies on exactly the same `termination_by` principles mapped to a sum type).*

Summary: Termination Checklist

Summary: Termination Checklist

When writing recursive algorithms in Lean 4:

1. Try to write it using **Structural Recursion**. Let Lean do the work.

Summary: Termination Checklist

When writing recursive algorithms in Lean 4:

1. Try to write it using **Structural Recursion**. Let Lean do the work.
2. If that fails, identify the mathematical **Measure** (remaining work) and use `termination_by`.

Summary: Termination Checklist

When writing recursive algorithms in Lean 4:

1. Try to write it using **Structural Recursion**. Let Lean do the work.
2. If that fails, identify the mathematical **Measure** (remaining work) and use `termination_by`.
3. Bind your `if` / `match` conditions to hypotheses (e.g., `h : a < b`) so the proof engine can "see" the branch logic.

Summary: Termination Checklist

When writing recursive algorithms in Lean 4:

1. Try to write it using **Structural Recursion**. Let Lean do the work.
2. If that fails, identify the mathematical **Measure** (remaining work) and use `termination_by`.
3. Bind your `if` / `match` conditions to hypotheses (e.g., `h : a < b`) so the proof engine can "see" the branch logic.
4. If dealing with multiple variables, order them carefully left-to-right to leverage **Lexicographic Priority**.

Summary: Termination Checklist

When writing recursive algorithms in Lean 4:

1. Try to write it using **Structural Recursion**. Let Lean do the work.
2. If that fails, identify the mathematical **Measure** (remaining work) and use `termination_by`.
3. Bind your `if` / `match` conditions to hypotheses (e.g., `h : a < b`) so the proof engine can "see" the branch logic.
4. If dealing with multiple variables, order them carefully left-to-right to leverage **Lexicographic Priority**.
5. When automated arithmetic fails, open a `decreasing_by` block and unleash `omega`.

Outline

Part 3: Algorithm Complexity and CSLib

Part 3: Algorithm Complexity & CSLib

Cost Models, Asymptotic Big-O, and Automated Complexity Proofs

Beyond Correctness: The Third Pillar

- In Section 1, we proved the algorithm computes the **Right Answer** (Correctness).

Beyond Correctness: The Third Pillar

- In Section 1, we proved the algorithm computes the **Right Answer** (Correctness).
- In Section 2, we proved the algorithm **Eventually Stops** (Termination).

Beyond Correctness: The Third Pillar

- In Section 1, we proved the algorithm computes the **Right Answer** (Correctness).
- In Section 2, we proved the algorithm **Eventually Stops** (Termination).
- **The Missing Piece:** *When* does it stop?

Beyond Correctness: The Third Pillar

- In Section 1, we proved the algorithm computes the **Right Answer** (Correctness).
- In Section 2, we proved the algorithm **Eventually Stops** (Termination).
- **The Missing Piece:** *When* does it stop?
- In mission-critical environments (e.g., Blockchain Smart Contracts, Real-time OS like seL4), a function that takes $O(2^n)$ time to terminate is functionally equivalent to an infinite loop. It is a Denial of Service (DoS) vulnerability.

Beyond Correctness: The Third Pillar

- In Section 1, we proved the algorithm computes the **Right Answer** (Correctness).
- In Section 2, we proved the algorithm **Eventually Stops** (Termination).
- **The Missing Piece:** *When* does it stop?
- In mission-critical environments (e.g., Blockchain Smart Contracts, Real-time OS like seL4), a function that takes $O(2^n)$ time to terminate is functionally equivalent to an infinite loop. It is a Denial of Service (DoS) vulnerability.
- Therefore, we must formally prove **Time and Space Complexity** as a mathematical theorem.

How to Count Steps in a Purely Functional Language?

How to Count Steps in a Purely Functional Language?

Lean has no side effects. We cannot simply mutate a global `counter++` variable inside a `while` loop.

How to Count Steps in a Purely Functional Language?

Lean has no side effects. We cannot simply mutate a global `counter++` variable inside a `while` loop.

There are two primary paradigms for Modeling Cost:

- **1. Intrinsic (The Monadic Approach):** Modify the algorithm to return a tuple `(Result, Cost)`. Every primitive operation adds to the cost. (Often implemented using a `Writer Monad`).

How to Count Steps in a Purely Functional Language?

Lean has no side effects. We cannot simply mutate a global `counter++` variable inside a `while` loop.

There are two primary paradigms for Modeling Cost:

- **1. Intrinsic (The Monadic Approach):** Modify the algorithm to return a tuple `(Result, Cost)`. Every primitive operation adds to the cost. (Often implemented using a `Writer Monad`).
- **2. Extrinsic (The Parallel Function):** Keep the original algorithm pure. Write a *second*, parallel function that mirrors the control flow of the algorithm but computes the steps taken.

How to Count Steps in a Purely Functional Language?

Lean has no side effects. We cannot simply mutate a global `counter++` variable inside a `while` loop.

There are two primary paradigms for Modeling Cost:

- **1. Intrinsic (The Monadic Approach):** Modify the algorithm to return a tuple `(Result, Cost)`. Every primitive operation adds to the cost. (Often implemented using a `Writer Monad`).
- **2. Extrinsic (The Parallel Function):** Keep the original algorithm pure. Write a *second*, parallel function that mirrors the control flow of the algorithm but computes the steps taken.
- In this seminar, we focus on the **Extrinsic** method, as it separates the complexity proof from the correctness proof.

Extrinsic Cost Modeling: An Example

Extrinsic Cost Modeling: An Example

Let's model the worst-case time complexity of inserting into a sorted list.

```
-- The pure algorithm
def insert (x : Nat) (xs : List Nat) : List Nat :=
  match xs with
  | [] => [x]
  | y :: ys => if x <= y then x :: y :: ys else y :: insert x ys

-- The parallel step-counting function
def time_insert (x : Nat) (xs : List Nat) : Nat :=
  match xs with
  | [] => 1                          -- 1 step for pattern match
  | y :: ys =>
      if x <= y then 2               -- 1 for match + 1 for comparison
      else 2 + time_insert x ys      -- Recursive cost addition
```

The Mathematical Foundation of Big-O in Lean

- To prove `time_insert` is $O(n)$, we need to define Big-O notation.

The Mathematical Foundation of Big-O in Lean

- To prove `time_insert` is $O(n)$, we need to define Big-O notation.
- In classical math: $f(n) = O(g(n))$ means $\exists c > 0, \exists N, \forall n \geq N, |f(n)| \leq c \cdot |g(n)|$.

The Mathematical Foundation of Big-O in Lean

- To prove `time_insert` is $O(n)$, we need to define Big-O notation.
- In classical math: $f(n) = O(g(n))$ means $\exists c > 0, \exists N, \forall n \geq N, |f(n)| \leq c \cdot |g(n)|$.
- **The Lean approach:** Mathlib does not hardcode Big-O for integers. Instead, it uses **Filters** (a concept from Bourbaki topology) to unify limits, infinity, and asymptotic behavior.

The Mathematical Foundation of Big-O in Lean

- To prove `time_insert` is $O(n)$, we need to define Big-O notation.
- In classical math: $f(n) = O(g(n))$ means $\exists c > 0, \exists N, \forall n \geq N, |f(n)| \leq c \cdot |g(n)|$.
- **The Lean approach:** Mathlib does not hardcode Big-O for integers. Instead, it uses **Filters** (a concept from Bourbaki topology) to unify limits, infinity, and asymptotic behavior.
- We use the filter `Filter.atTop` to represent "as the variable tends to infinity".

The Mathematical Foundation of Big-O in Lean

- To prove `time_insert` is $O(n)$, we need to define Big-O notation.
- In classical math: $f(n) = O(g(n))$ means $\exists c > 0, \exists N, \forall n \geq N, |f(n)| \leq c \cdot |g(n)|$.
- **The Lean approach:** Mathlib does not hardcode Big-O for integers. Instead, it uses **Filters** (a concept from Bourbaki topology) to unify limits, infinity, and asymptotic behavior.
- We use the filter `Filter.atTop` to represent "as the variable tends to infinity".
- **Syntax:** `Asymptotics.IsBigO f g Filter.atTop`

The Mathematical Foundation of Big-O in Lean

- To prove `time_insert` is $O(n)$, we need to define Big-O notation.
- In classical math: $f(n) = O(g(n))$ means $\exists c > 0, \exists N, \forall n \geq N, |f(n)| \leq c \cdot |g(n)|$.
- **The Lean approach:** Mathlib does not hardcode Big-O for integers. Instead, it uses **Filters** (a concept from Bourbaki topology) to unify limits, infinity, and asymptotic behavior.
- We use the filter `Filter.atTop` to represent "as the variable tends to infinity".
- **Syntax:** `Asymptotics.IsBigO f g Filter.atTop`
- **Notation:** `f =O[atTop] g`

Stating a Complexity Theorem

Stating a Complexity Theorem

Because Big-O operates on functions of type `Nat -> Nat` (or Reals), we must abstract our `time_insert` over the *size* of the input.

```
-- 1. Define the worst-case time over the size 'n'
def T_insert (n : Nat) : Nat :=
  -- mathematically, the worst case of time_insert is 2*n + 1
  2 * n + 1

-- 2. State the Big-O Theorem
open Asymptotics Filter

theorem insert_is_O_n :
  T_insert =O[atTop] (fun n => n) := by
  -- Proof involves showing there exists a constant c
  -- such that 2n + 1 <= c * n for large n.
  ...
```

The Problem with Manual Complexity Proofs

The Problem with Manual Complexity Proofs

While proving $O(n)$ is trivial, proving complex recurrences is a nightmare.

- Consider Merge Sort: $T(n) = 2T(n/2) + O(n)$.

The Problem with Manual Complexity Proofs

While proving $O(n)$ is trivial, proving complex recurrences is a nightmare.

- Consider Merge Sort: $T(n) = 2T(n/2) + O(n)$.
- Manually unwinding this recurrence relation in Lean requires handling floor/ceiling functions for odd n, proving bounds at each tree depth, and summing geometric series manually.

The Problem with Manual Complexity Proofs

While proving $O(n)$ is trivial, proving complex recurrences is a nightmare.

- Consider Merge Sort: $T(n) = 2T(n/2) + O(n)$.
- Manually unwinding this recurrence relation in Lean requires handling floor/ceiling functions for odd n, proving bounds at each tree depth, and summing geometric series manually.
- If we change one line of code, we have to rewrite a 500-line asymptotic proof.

The Problem with Manual Complexity Proofs

While proving $O(n)$ is trivial, proving complex recurrences is a nightmare.

- Consider Merge Sort: $T(n) = 2T(n/2) + O(n)$.
- Manually unwinding this recurrence relation in Lean requires handling floor/ceiling functions for odd n, proving bounds at each tree depth, and summing geometric series manually.
- If we change one line of code, we have to rewrite a 500-line asymptotic proof.
- **We need specialized infrastructure.** Enter **CSLib**.

Introducing CSLib: The Missing Piece

- **What is Mathlib?** The standard library for pure mathematics (Algebra, Topology, Measure Theory). It is excellent for continuous math but lacks deep support for discrete algorithm analysis.

Introducing CSLib: The Missing Piece

- **What is Mathlib?** The standard library for pure mathematics (Algebra, Topology, Measure Theory). It is excellent for continuous math but lacks deep support for discrete algorithm analysis.
- **What is CSLib?** The emerging ecosystem designed to be the "Mathlib for Computer Science".

Introducing CSLib: The Missing Piece

- **What is Mathlib?** The standard library for pure mathematics (Algebra, Topology, Measure Theory). It is excellent for continuous math but lacks deep support for discrete algorithm analysis.
- **What is CSLib?** The emerging ecosystem designed to be the "Mathlib for Computer Science".
- **Core Components of CSLib:**

Introducing CSLib: The Missing Piece

- **What is Mathlib?** The standard library for pure mathematics (Algebra, Topology, Measure Theory). It is excellent for continuous math but lacks deep support for discrete algorithm analysis.
- **What is CSLib?** The emerging ecosystem designed to be the "Mathlib for Computer Science".
- **Core Components of CSLib:**
  - **Data Structures:** Pre-verified, highly optimized structures (Red-Black Trees, Hash Maps) with intrinsic complexity theorems.

Introducing CSLib: The Missing Piece

- **What is Mathlib?** The standard library for pure mathematics (Algebra, Topology, Measure Theory). It is excellent for continuous math but lacks deep support for discrete algorithm analysis.
- **What is CSLib?** The emerging ecosystem designed to be the "Mathlib for Computer Science".
- **Core Components of CSLib:**
  - **Data Structures:** Pre-verified, highly optimized structures (Red-Black Trees, Hash Maps) with intrinsic complexity theorems.
  - **Computation Models:** Formalized Turing Machines, RAM models, and Lambda Calculus.

Introducing CSLib: The Missing Piece

- **What is Mathlib?** The standard library for pure mathematics (Algebra, Topology, Measure Theory). It is excellent for continuous math but lacks deep support for discrete algorithm analysis.
- **What is CSLib?** The emerging ecosystem designed to be the "Mathlib for Computer Science".
- **Core Components of CSLib:**
  - **Data Structures:** Pre-verified, highly optimized structures (Red-Black Trees, Hash Maps) with intrinsic complexity theorems.
  - **Computation Models:** Formalized Turing Machines, RAM models, and Lambda Calculus.
  - **Imperative Verification:** Separation Logic frameworks for verifying C-like code with pointers and memory mutations.

The Frontier: Automated Complexity Proofs

The Frontier: Automated Complexity Proofs

Modern verification (like the tools bundled in or inspired by CSLib) aims to completely **automate** the extraction and proof of recurrences.

- **Step 1: Syntactic Extraction.** Using Lean 4's powerful macro system, we can write a metaprogram that reads the AST of your `def` and automatically generates the `time_` function.

The Frontier: Automated Complexity Proofs

Modern verification (like the tools bundled in or inspired by CSLib) aims to completely **automate** the extraction and proof of recurrences.

- **Step 1: Syntactic Extraction.** Using Lean 4's powerful macro system, we can write a metaprogram that reads the AST of your `def` and automatically generates the `time_` function.
- **Step 2: Recurrence Solving.** Instead of manual induction, the library provides a formalized **Master Theorem** (or Akra-Bazzi method).

The Frontier: Automated Complexity Proofs

Modern verification (like the tools bundled in or inspired by CSLib) aims to completely **automate** the extraction and proof of recurrences.

- **Step 1: Syntactic Extraction.** Using Lean 4's powerful macro system, we can write a metaprogram that reads the AST of your `def` and automatically generates the `time_` function.
- **Step 2: Recurrence Solving.** Instead of manual induction, the library provides a formalized **Master Theorem** (or Akra-Bazzi method).
- **Step 3: Tactic Integration.** A custom tactic (e.g., `complexity`) matches the extracted recurrence against the Master Theorem database and discharges the proof instantly.

What Automated Complexity Looks Like (Conceptual)

What Automated Complexity Looks Like (Conceptual)

With modern CS libraries, the boilerplate vanishes. The user experience approaches the simplicity of writing a type signature.

```
-- We annotate the function to automatically generate the cost model
@[derive_cost]
def merge_sort (xs : List Nat) : List Nat :=
  ... -- standard merge sort implementation

-- We state the theorem using a custom auto-tactic
theorem merge_sort_complexity :
  cost_merge_sort =O[atTop] (fun n => n * log n) := by
  -- The tactic extracts the recurrence T(n) = 2T(n/2) + O(n)
  -- applies the formalized Master Theorem, and closes the goal.
  auto_bound
```

What Automated Complexity Looks Like (Conceptual)

With modern CS libraries, the boilerplate vanishes. The user experience approaches the simplicity of writing a type signature.

```
-- We annotate the function to automatically generate the cost model
@[derive_cost]
def merge_sort (xs : List Nat) : List Nat :=
  ... -- standard merge sort implementation

-- We state the theorem using a custom auto-tactic
theorem merge_sort_complexity :
  cost_merge_sort =O[atTop] (fun n => n * log n) := by
  -- The tactic extracts the recurrence T(n) = 2T(n/2) + O(n)
  -- applies the formalized Master Theorem, and closes the goal.
  auto_bound
```

Note: This is the active research frontier. Tools like `AutoBound` and `Wormhole` are actively bridging the gap between theoretical CS and Lean's automation.

Summary of Section 3

Summary of Section 3

1. **Complexity is Correctness:** Avoiding asymptotic explosion is a strict requirement for verified systems.

Summary of Section 3

1. **Complexity is Correctness:** Avoiding asymptotic explosion is a strict requirement for verified systems.
2. **Extrinsic Cost Models:** We mathematically mirror algorithms with parallel functions that count execution steps.

Summary of Section 3

1. **Complexity is Correctness:** Avoiding asymptotic explosion is a strict requirement for verified systems.
2. **Extrinsic Cost Models:** We mathematically mirror algorithms with parallel functions that count execution steps.
3. **Filters for Big-O:** Lean represents limits and asymptotic bounds using the rigorous `Filter.atTop` framework (`f =O[atTop] g`).

Summary of Section 3

1. **Complexity is Correctness:** Avoiding asymptotic explosion is a strict requirement for verified systems.
2. **Extrinsic Cost Models:** We mathematically mirror algorithms with parallel functions that count execution steps.
3. **Filters for Big-O:** Lean represents limits and asymptotic bounds using the rigorous `Filter.atTop` framework (`f =O[atTop] g`).
4. **CSLib Ecosystem:** Provides the required theorems (like the Master Theorem) and verified data structures that pure Mathlib lacks.

Summary of Section 3

1. **Complexity is Correctness:** Avoiding asymptotic explosion is a strict requirement for verified systems.
2. **Extrinsic Cost Models:** We mathematically mirror algorithms with parallel functions that count execution steps.
3. **Filters for Big-O:** Lean represents limits and asymptotic bounds using the rigorous `Filter.atTop` framework (`f =O[atTop] g`).
4. **CSLib Ecosystem:** Provides the required theorems (like the Master Theorem) and verified data structures that pure Mathlib lacks.
5. **Automation is Key:** Metaprogramming in Lean 4 allows us to extract cost recurrences and solve them automatically, hiding the tedious math from the software engineer.

Final Project Framework

Final Project Framework

Designing an Automated Complexity Prover using the `TimeM` Monad

Project Motivation: Intrinsic vs. Extrinsic

- In Section 3, we explored the **Extrinsic** approach: writing a pure function alongside a completely separate `time_` function.

Project Motivation: Intrinsic vs. Extrinsic

- In Section 3, we explored the **Extrinsic** approach: writing a pure function alongside a completely separate `time_` function.
- **The Drawback:** Maintaining two parallel functions is error-prone. If you optimize the main algorithm, you must remember to manually update the tracking function, or your proofs break.

Project Motivation: Intrinsic vs. Extrinsic

- In Section 3, we explored the **Extrinsic** approach: writing a pure function alongside a completely separate `time_` function.
- **The Drawback:** Maintaining two parallel functions is error-prone. If you optimize the main algorithm, you must remember to manually update the tracking function, or your proofs break.
- **The Solution:** The **Intrinsic** approach. We embed step-counting directly into the execution context of the function using a custom Monad.

Project Motivation: Intrinsic vs. Extrinsic

- In Section 3, we explored the **Extrinsic** approach: writing a pure function alongside a completely separate `time_` function.
- **The Drawback:** Maintaining two parallel functions is error-prone. If you optimize the main algorithm, you must remember to manually update the tracking function, or your proofs break.
- **The Solution:** The **Intrinsic** approach. We embed step-counting directly into the execution context of the function using a custom Monad.
- **Your Final Project:** You will implement a step-counting monad called `TimeM`, write standard algorithms within it, and build a verification pipeline to automate their asymptotic complexity proofs.

Defining the `TimeM` Monad

Defining the `TimeM` Monad

Under the hood, `TimeM` is simply a state monad that carries an accumulating natural number representing "ticks" or execution cost.

```
-- Define the stateful computation type
def TimeM (A : Type) : Type := Nat -> A * Nat

-- Making it a Monad so we can use do-notation
instance : Monad TimeM where
  pure x := fun c => (x, c)
  bind x f := fun c =>
    let (res, c') := x c
    f res c'
```

Defining the `TimeM` Monad

Under the hood, `TimeM` is simply a state monad that carries an accumulating natural number representing "ticks" or execution cost.

```
-- Define the stateful computation type
def TimeM (A : Type) : Type := Nat -> A * Nat

-- Making it a Monad so we can use do-notation
instance : Monad TimeM where
  pure x := fun c => (x, c)
  bind x f := fun c =>
    let (res, c') := x c
    f res c'
```

We define a foundational primitive operation to increment our clock:

```
def tick : TimeM Unit :=
  fun c => ((), c + 1)
```

Writing Algorithms inside `TimeM`

Writing Algorithms inside `TimeM`

Using Lean's do-notation, we can write algorithms that look identical to standard imperative code, explicitly instrumenting our cost metrics.

```
def insertM (x : Nat) (xs : List Nat) : TimeM (List Nat) :=
  match xs with
  | [] => return [x]
  | y :: ys => do
      tick -- Charge 1 cost for the comparison operation
      if x <= y then
        return x :: y :: ys
      else do
        let rest <- insertM x ys
        return y :: rest
```

Extracting the Cost Function

Extracting the Cost Function

To verify the execution cost, we need to evaluate the monad starting from an initial clock cycle of 0, discarding the computed output value.

```
-- Helper to extract ONLY the accumulated cost
def getCost {A : Type} (comp : TimeM A) : Nat :=
  (comp 0).2

-- Example evaluation
#eval getCost (insertM 5 [1, 2, 3, 4]) -- Outputs: 4
```

Extracting the Cost Function

To verify the execution cost, we need to evaluate the monad starting from an initial clock cycle of 0, discarding the computed output value.

```
-- Helper to extract ONLY the accumulated cost
def getCost {A : Type} (comp : TimeM A) : Nat :=
  (comp 0).2

-- Example evaluation
#eval getCost (insertM 5 [1, 2, 3, 4]) -- Outputs: 4
```

The Mathematical Target: For any list `xs`, we want to formally prove an upper bound on `getCost (insertM x xs)` proportional to the list's length.

Live Lean: `D6_TimeM.lean`

Project Milestone 1: Manual Monadic Bounds

Project Milestone 1: Manual Monadic Bounds

Your first milestone is to manually prove that the monadic implementation satisfies a precise mathematical bound before mapping it to Big-O.

```
theorem insertM_cost_bound (x : Nat) (xs : List Nat) :
  getCost (insertM x xs) <= xs.length := by
  induction xs with
  | nil =>
    -- Base case: length is 0, cost is 0
    rfl
  | cons y ys ih =>
    -- Inductive step: unfolds the match and uses
    -- the inductive hypothesis (ih)
    unfold insertM getCost
    ...
```

Project Milestone 2: Automation Blueprint

Project Milestone 2: Automation Blueprint

Manually writing inductive proofs for every cost bound becomes redundant. The core goal of this project is to build an automated workflow.

- **Goal:** Create a macro or a unified helper tactic that can instantly discharge basic complexity bounds for structural steps.

Project Milestone 2: Automation Blueprint

Manually writing inductive proofs for every cost bound becomes redundant. The core goal of this project is to build an automated workflow.

- **Goal:** Create a macro or a unified helper tactic that can instantly discharge basic complexity bounds for structural steps.
- **Mechanism:** Monadic code written with simple structural recursion creates highly predictable cost equations during unfolding.

Project Milestone 2: Automation Blueprint

Manually writing inductive proofs for every cost bound becomes redundant. The core goal of this project is to build an automated workflow.

- **Goal:** Create a macro or a unified helper tactic that can instantly discharge basic complexity bounds for structural steps.
- **Mechanism:** Monadic code written with simple structural recursion creates highly predictable cost equations during unfolding.
- You will compile a hint database of lemmas showing how `tick` interacts with `bind`, allowing Lean's simplifier (`simp`) to flatten monadic state tracking automatically.

Designing the Auto-Proof Pipeline

Designing the Auto-Proof Pipeline

By combining structural induction with a customized rewriting sequence, we can create a composite tactic sequence that solves these goals seamlessly.

```
-- Conceptual pipeline for your custom automation
macro "prove_complexity" : tactic => `(tactic|
  intros;
  macro_rules | `(tactic| leaves) => `(tactic| rfl)
  induction_by_containers; -- custom container unroller
  simp [getCost, insertM, tick];
  omega
)
```

Designing the Auto-Proof Pipeline

By combining structural induction with a customized rewriting sequence, we can create a composite tactic sequence that solves these goals seamlessly.

```
-- Conceptual pipeline for your custom automation
macro "prove_complexity" : tactic => `(tactic|
  intros;
  macro_rules | `(tactic| leaves) => `(tactic| rfl)
  induction_by_containers; -- custom container unroller
  simp [getCost, insertM, tick];
  omega
)
```

Your project must demonstrate that this automated tactic can successfully close the complexity theorems for at least three distinct algorithms: Linear Search, Insertion Sort, and Binary Tree lookup.

Project Deliverables and Grading Criteria

To earn full credit on the final project, your submission must include:

Project Deliverables and Grading Criteria

To earn full credit on the final project, your submission must include:

1. **The Core Framework:** A fully operational definition of the `TimeM` monad instantiation and its corresponding evaluation infrastructure.

Project Deliverables and Grading Criteria

To earn full credit on the final project, your submission must include:

1. **The Core Framework:** A fully operational definition of the `TimeM` monad instantiation and its corresponding evaluation infrastructure.
2. **The Monadic Library:** Implementations of Linear Search, Insertion Sort, and Matrix Multiplication using the `TimeM` paradigm.

Project Deliverables and Grading Criteria

To earn full credit on the final project, your submission must include:

1. **The Core Framework:** A fully operational definition of the `TimeM` monad instantiation and its corresponding evaluation infrastructure.
2. **The Monadic Library:** Implementations of Linear Search, Insertion Sort, and Matrix Multiplication using the `TimeM` paradigm.
3. **The Automation Engine:** A custom Lean 4 macro or proof script that handles structural cost simplification without manual intervention per branch.

Project Deliverables and Grading Criteria

To earn full credit on the final project, your submission must include:

1. **The Core Framework:** A fully operational definition of the `TimeM` monad instantiation and its corresponding evaluation infrastructure.
2. **The Monadic Library:** Implementations of Linear Search, Insertion Sort, and Matrix Multiplication using the `TimeM` paradigm.
3. **The Automation Engine:** A custom Lean 4 macro or proof script that handles structural cost simplification without manual intervention per branch.
4. **Asymptotic Integration:** Final proofs framing your extracted monadic costs within Mathlib's asymptotic Big-O filter definitions (`=O[atTop]`).